
INTERMEDIATE ARTIFACTS AS FIRST-CLASS CITIZENS: A DATA MODEL FOR PERSISTENT LLM REASONING

Josh Rosen
ThruWire, Inc.

Seth Rosen
ThruWire, Inc.

May 7, 2026

ABSTRACT

Many important AI workflows are multi-step, revisable, and long-lived, yet most current systems still optimize around prompts, transcripts, and final answers. Intermediate work is often discarded, overwritten, summarized into prose, or left only inside execution-local reasoning traces. This makes it difficult to inspect how a result was formed, selectively update only the affected parts of a workflow, preserve stable upstream work, or maintain coherent state across repeated revisions.

This paper argues that intermediate artifacts should be treated as the primary unit of *persistent* reasoning state for long-lived AI systems. We introduce an artifact-centric data model in which intermediate outputs are deliberate, typed, addressable, versioned, dependency-aware objects that can be consumed by downstream nodes. In this view, final answers are projections over a maintained artifact graph rather than the sole durable product of reasoning, while transcripts and hidden reasoning remain complementary parts of the overall system.

We position this model as the data-model counterpart to execution lineage. Prior work on execution lineage asks how AI-native work should execute; this paper asks what durable state that execution should produce. We distinguish artifact lineage from prompt lineage and execution lineage, formalize additive and superseding update semantics, and show why graph execution requires a corresponding artifact model if replay, recomputation, revision, and iterative inspection and improvement by clients are to remain intelligible.

The contribution is a systems-level abstraction, not a claim that structured artifacts “solve reasoning,” replace hidden chain-of-thought, or remove model stochasticity. Rather, we argue that improving persistent LLM reasoning requires a durable artifact substrate outside the model, with explicit semantics for identity, lineage, supersession, downstream consumption, and iterative inspection and improvement by clients.

1 Introduction

Large language model systems are often used for more than single-turn question answering. In many important settings they support extended work: research, analysis, planning, transformation, drafting, revision, and coordination across tools and collaborators. In such settings, the operational question is no longer only whether a model can produce a plausible answer now. It is whether the surrounding system can preserve useful intermediate state, revise it coherently later, and explain what changed, what did not, and why.

Most current LLM interfaces do not treat this as a first-class systems problem. They privilege final answers. Agent loops may expose some intermediate text, but that intermediate work is often fragile. It can be buried in transcripts, rewritten wholesale at the next turn, reduced to summaries, or left only inside private chain-of-thought or controller state. As a result, the parts of the process that would be most useful to inspect and improve often evaporate after execution.

This is a poor fit for long-lived AI-native work. A multi-step workflow that produces a research memo, decision matrix, implementation plan, and revised recommendation is not well modeled as one prompt plus one answer. It produces a maintained body of intermediate state. If that state is disposable, downstream work becomes hard to inspect and

hard to update. If a constraint changes upstream, the system often has no principled way to know which downstream outputs should be preserved, which should be recomputed, and which prior artifacts have been replaced.

Prior work introduced *execution lineage* as an execution model for AI-native work represented as deterministic graphs of artifact-producing computation. Its central question was: *how should such work execute?* The present paper addresses the complementary question: *what durable state should that execution produce?* The answer we defend is that intermediate artifacts should be treated as first-class citizens of the system. They are not merely logs, not only UI affordances, and not simply cached prompt fragments. They are typed system state.

Our thesis is straightforward:

LLM systems should treat selected intermediate outputs as structured, persistent, addressable state so that reasoning can be replayed, inspected, improved, and iterated on across time.

This thesis has several parts. First, chat transcripts are useful for audit, narration, and interaction, but they are poor computational boundaries. Second, chain-of-thought and other hidden reasoning traces may help a model reason locally [8, 11], and future systems will likely continue to rely on such internal state. But those traces are typically execution-local, untyped, and hard to validate or revise directly. Third, final answers are not enough. In long-lived systems, the final answer is often only a projection or synthesis over a deeper maintained state that includes evidence digests, claim matrices, criteria, plans, analyses, and revisions. What persistent systems need is not to eliminate hidden reasoning, but to complement it with deliberate intermediate artifacts that can be inspected, revised, and reused.

The distinction matters because persistent work requires semantics that prompt-centric systems usually lack. Intermediate artifacts must be addressable, versioned, dependency-aware, and consumable by downstream nodes. A graph execution model requires an artifact model: if nodes and steps have explicit boundaries, then their outputs must also have explicit boundaries. Without that, execution lineage degrades back into prompt lineage because downstream computation still depends on loosely reconstructed textual context rather than durable state.

The paper makes four contributions. First, it formulates an artifact-centric data model for persistent LLM reasoning and defines the properties that make an artifact first-class. Second, it distinguishes *artifact lineage* from *prompt lineage* and *execution lineage*. Third, it introduces additive versus superseding update semantics as a necessary mechanism for maintaining coherent artifact state over time. Fourth, it argues conceptually that evaluating only final outputs misses the primary failure mode of long-lived AI systems: maintained-state degradation.

Our claim is deliberately narrow and system-level. We do not claim model-level determinism, nor do we claim that every task requires durable artifacts. Some short-horizon conversational tasks do not. We also do not claim that schemas or structured artifacts are free; they require discipline, careful boundary design, and explicit tradeoffs against flexibility. The claim is instead that once AI work becomes multi-step, revisable, and shared across time, persistent progress depends on intermediate artifacts that sit alongside transcripts, hidden reasoning, and final outputs as the maintainable state surface.

2 Motivating Example

Consider a policy-analysis workflow that produces a recommendation memo about telehealth expansion. A plausible execution includes source digests for utilization, reimbursement, operations, and access-cost tradeoffs; a claim matrix grounded in those sources; a tension analysis that preserves unresolved conflicts; a recommendation-criteria artifact; an implementation plan; and finally a memo.

In a transcript-centric system, this work often collapses into a conversational loop. The model sees prior notes, current sources, and perhaps the previous memo. When a new constraint arrives, such as a requirement that year-one expansion be budget-neutral, the common behavior is to regenerate a revised memo. Some systems may also produce a new summary or update a scratch note. But the core maintained state is weak. The earlier claim matrix may no longer align with the updated criteria. The implementation plan may mention scaling steps that the revised memo now implicitly forbids. A future update inherits this partially inconsistent state.

This is not only an inconvenience for human readers. It is a systems failure. The workflow already had meaningful intermediate boundaries: evidence summaries, claims, tensions, criteria, and plan. Those boundaries should have been first-class artifacts. If the criteria artifact is revised, the system should know that the criteria artifact has changed, that the implementation plan and final memo are downstream dependents, that upstream evidence artifacts remain valid, and that unaffected branches should be preserved exactly. If an adjacent staffing note is updated but is not upstream of the final memo, the memo should remain unchanged.

The key point is that the relevant state here is not only the final memo. The maintained body of work is the substrate on which future revisions depend. A workflow that stores only a transcript plus a final answer has lost the very state required for reliable evolution.

Figure 1 contrasts these two views. On the left, a loop collapses intermediate work into an answer-oriented transcript. On the right, an artifact-first workflow preserves typed intermediate state, supports revision via explicit supersession, and makes downstream dependencies addressable.

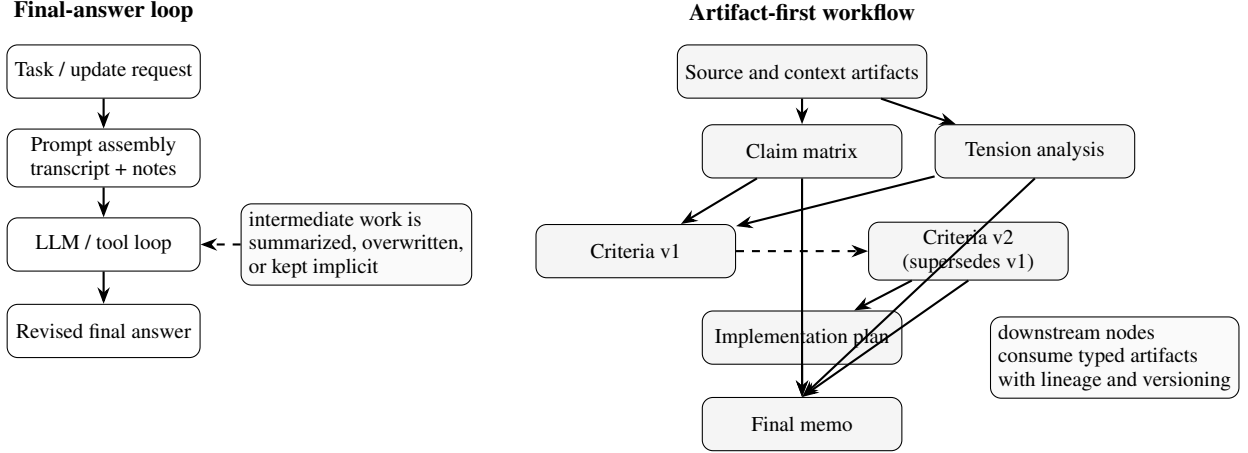


Figure 1: A final-answer loop collapses intermediate state into a transcript and answer. An artifact-first workflow preserves typed intermediate artifacts, allows explicit supersession, and keeps downstream dependencies addressable.

3 Background and Related Work

3.1 Agent Loops, Tool Use, and Intermediate Reasoning

Agentic LLM systems commonly interleave reasoning, tool use, and revision inside iterative loops. ReAct [1] remains the canonical example of this paradigm, while Toolformer [2], AutoGen [6], CAMEL [3], Voyager [4], and MetaGPT [5] explore richer tool ecosystems, role structures, and multi-agent coordination. These systems demonstrate that substantial capability gains can be achieved by placing models inside structured harnesses rather than treating them as isolated text generators.

Work on chain-of-thought, scratchpads, self-refinement, and search further shows that intermediate steps can materially improve performance [7, 8, 9, 10, 11, 12, 13, 14]. This literature is directly relevant because it establishes the practical value of intermediate reasoning. At the same time, most of it still treats those intermediates as execution-local textual traces. Our argument is not against intermediate reasoning; it is that long-lived systems need an explicit substrate for selected intermediate state outside the transient reasoning trace.

3.2 Harnesses, Workflow Memory, and Execution Substrates

Recent work increasingly treats the agent harness itself as an object of study. Natural-Language Agent Harnesses [15] and modular harness research [16] argue that important behavior lives in controller logic, memory, and interfaces around the model. Memory-oriented systems such as Agent Workflow Memory [17], WorkflowLLM [18], LEGOMem [19], Memory-R1 [20], and Memori [21] likewise recognize that long-lived AI work depends on persistent state and retrieval.

These lines of work are complementary to ours but not identical. A memory system decides what can be retained and later retrieved. An artifact model specifies what kind of durable state the system should persist in the first place, what semantics govern its replacement, and how downstream computation should consume it. In that sense, our focus is less on retrieval policy than on state representation.

3.3 DAG Workflows, Data Lineage, and Provenance

Workflow and data systems have long relied on explicit dependency graphs, materialized intermediates, and lineage-aware recomputation. Dryad [22], Spark [23], Airflow [24], and dbt [25] all reflect a mature systems lesson: once work becomes staged and collaborative, hidden dependencies and ephemeral outputs become liabilities. The same general lesson appears in data provenance research, including why/where provenance [26] and scientific-workflow provenance systems such as VisTrails [27]. Provenance-aware LLM-agent work in scientific workflows [28] extends that concern into the agent era.

Our proposal differs in target object. Classical provenance systems usually record how deterministic data products were produced. LLM workflows add stochastic generation, prompt-conditioned steps, and richer intermediate semantics. The need for lineage remains, but the durable state is not only tables and files; it includes intermediate analytical artifacts intended for both humans and downstream model steps.

3.4 Notebooks, Literate Workflows, and Intermediate Representations

The importance of preserving intermediate work is not unique to AI systems. Literate programming [29] and notebooks [30] both treat intermediate computational state as a meaningful object of inspection and revision rather than a hidden implementation detail. Similarly, compiler design depends on explicit intermediate representations because they create analyzable, transformable, and optimizable boundaries between stages [31]. These traditions matter here because they provide precedents for two claims we make: first, that intermediate state is often more useful than an opaque end product; and second, that good systems expose explicit intermediate forms instead of forcing downstream work to parse free-form history.

3.5 Why Knowledge Graphs and Wikis Are Not Enough

Knowledge graphs and wikis preserve facts and relations, and they are valuable long-term memory surfaces. But they are not sufficient as a general substrate for persistent reasoning. Knowledge-graph systems usually capture static semantic relationships rather than execution identity, procedural dependencies, or invalidation semantics [32]. Wikis can store notes, but they do not ordinarily specify which downstream outputs should be recomputed when an upstream artifact changes. An artifact-centric reasoning system needs not only stored knowledge but also regeneration structure.

4 Problem Formulation

We focus on workflows whose outputs are revised over time rather than consumed only once. Let a workflow be a directed acyclic graph $G = (V, E)$ in which each node or step produces some output that may be consumed later. The usual question is how to execute G reproducibly. Here we ask a different question: what should be persisted at each boundary so that the workflow’s state can be inspected, reused, updated, and recomputed coherently?

Transcript-centric systems implicitly answer: persist the conversation history, maybe a summary, and perhaps the final answer. This answer is insufficient for at least four reasons.

- **Inspection failure.** A transcript is not a clean unit of state. It mixes tasking, reasoning, tool observations, and multiple candidate outputs.
- **Reuse failure.** Downstream consumers must heuristically extract the relevant part of the transcript or depend on an entire prior answer.
- **Revision failure.** The system lacks principled semantics for replacing one intermediate result while preserving unaffected state.
- **Evaluation failure.** Final-output metrics can look good even when intermediate maintained state has drifted or become inconsistent.

The underlying issue is a mismatch between execution structure and state structure. A graph execution model creates nodes, dependencies, replay boundaries, and recomputation scope. But if the outputs of those nodes are flattened back into prompt context, then the graph exists only at execution time. The durable state remains transcript-shaped. This undermines the very benefits the graph was meant to provide.

We therefore define persistent reasoning as a systems property in which selected intermediate outputs are represented as durable artifacts with explicit semantics for identity, dependency, and revision. This is complementary to hidden chain-of-thought. Hidden chain-of-thought, when present, is private execution-local reasoning. Persistent artifacts are deliberate outputs published at explicit boundaries for later review, improvement, or reuse.

5 Artifact-Centric Reasoning Model

5.1 Artifacts as First-Class State

An artifact-centric reasoning system treats intermediate outputs as first-class state when they satisfy six properties.

Typed. An artifact has an explicit role or schema surface. The type may be lightweight, such as a structured memo section or decision matrix, or more rigid, such as a JSON payload. The point is not maximal formality; it is that downstream consumers should know what kind of object they are reading.

Structured. The artifact has internal organization beyond arbitrary free text. Structure may be tabular, sectioned, key-value, XML-like, or otherwise canonicalized. ThruWire’s current implementation, for example, distinguishes artifact roles from artifact types and supports renderer-facing structured payloads, including partial artifact construction and rendering during streaming. The paper’s claim is the abstraction behind that implementation: artifacts should support reliable downstream consumption and validation.

Addressable. The artifact can be referred to directly, rather than only by replaying the conversation that happened to produce it. Addressability is the precondition for reuse, inspection, and comparison across runs.

Versioned. The system can distinguish an earlier artifact from a later one. Versioning does not require user-facing version numbers in every case; it requires identity surfaces that let the system know when an artifact is the same, new, or revised.

Dependency-aware. The artifact records which upstream state it depends on. This is what allows downstream invalidation and recomputation to be principled rather than heuristic.

Consumable downstream. The artifact is not persisted only for audit. It is an operational dependency surface for other nodes or steps.

Together these properties define a first-class artifact as more than a saved message. It is a typed unit of durable system state.

5.2 Intermediate Versus Final Artifacts

An artifact-centric model distinguishes final artifacts from intermediate artifacts, but it does not treat the final answer as the only durable object. The final answer is often a projection over maintained state: a synthesis of upstream evidence, criteria, and plan. By contrast, intermediate artifacts preserve the reasoning substrate on which future work depends.

This distinction has methodological consequences. If evaluation measures only the final answer, then a workflow that rewrites its memo convincingly but leaves stale criteria or inconsistent plan artifacts may appear successful. Yet its maintained state is already degraded. Long-lived AI work therefore requires evaluation of *maintained-state quality*, not only final-answer quality.

5.3 Artifact Payloads and Deliberate Publication

First-class artifacts should be published deliberately. A system may internally generate candidate drafts, narration, or step-local traces, but only selected artifacts become authoritative downstream state. This publication boundary matters. It distinguishes a durable artifact from raw private reasoning. It also separates two concerns that are often conflated:

- **How the model reasoned locally**
- **What state the system chose to publish and maintain**

The first may remain private, partial, or non-canonical. The second must be reviewable and fit for downstream use.

6 Artifact Lineage and Supersession Semantics

6.1 Three Forms of Lineage

Prior work on execution lineage established the lineage of computation that produced an output under explicit graph execution. For the present paper, it is useful to distinguish three related but non-identical kinds of lineage.

Prompt lineage. Prompt lineage records what text, instructions, and transcript state led to a model response. It is useful for audit and debugging, but it is not a stable substrate for composition.

Execution lineage. Execution lineage records what node or step executed, under what resolved inputs and dependency identities, and what computation was replayed or recomputed.

Artifact lineage. Artifact lineage records what durable state was produced, consumed, revised, or superseded. It is concerned with maintained objects rather than prompts or runs alone.

These lineages answer different questions. Prompt lineage answers “what text led to this?” Execution lineage answers “what computation ran?” Artifact lineage answers “what durable state exists now, what did it replace, and what downstream state depends on it?”

6.2 Additive Versus Superseding Semantics

Persistent reasoning systems need a clear distinction between additive and superseding updates.

Additive semantics. Some steps produce new artifacts without invalidating earlier ones. A new research note, an additional scenario branch, or a fresh evidence extract may coexist with prior artifacts. In additive updates, earlier artifacts remain valid and addressable.

Superseding semantics. Other steps revise or replace prior artifacts. A current reimbursement summary may supersede a stale version; a revised decision criterion may supersede an earlier criterion; a corrected claim matrix may supersede the earlier matrix it replaces. In these cases, leaving all versions equally active would make the maintained state incoherent.

Supersession is therefore not simply “another artifact exists.” It is a semantic relation stating that one artifact replaces another as the current authoritative state for some role. The earlier artifact may remain in history for audit, but it should no longer be treated as active current state for downstream consumers.

This distinction is especially important when artifacts are structured payloads rather than undifferentiated prose. Without supersession semantics, downstream consumers may combine incompatible versions. Without additive semantics, the system loses the ability to preserve alternative branches or cumulative notes that were never meant to invalidate prior work.

6.3 State Quality as a First-Class Evaluation Target

The additive-versus-superseding distinction also clarifies why state quality deserves separate evaluation. A system can often repair the visible final answer even when it has lost track of which intermediate objects are still current. For example, a loop may incorporate a new budget-neutrality constraint into the final memo while leaving an older implementation plan or recommendation-criteria note effectively active in surrounding state. The human may not notice immediately because the final answer looks coherent in isolation. Yet the system has already accumulated state debt: later revisions inherit artifacts whose authority is ambiguous.

This is the artifact-centric analogue of stale cache invalidation in other computing systems. The main difference is that the stale objects here are not only cached computations; they are reasoning-state objects that future humans or models may read as if they were still current. A serious evaluation framework must therefore test not only whether the final answer changed appropriately, but whether the maintained artifact set now reflects a coherent “current state” after revision.

6.4 Formal Intuition

Let A be the set of artifacts in a workflow state. Each artifact $a \in A$ has at least:

$$a = (\tau, \iota, D, p, s) \tag{1}$$

where τ is a type or role, ι is an identity, D is a dependency set, p is the payload, and s is a status surface indicating whether the artifact is active, historical, or superseded.

An additive step creates a new artifact a' without changing the active status of existing artifacts in the same role. A superseding step creates a' and updates the status of at least one prior artifact a so that downstream consumers know a' is now authoritative for that role. This need not imply deletion. In fact, preserving the superseded artifact often improves auditability and replay. The key is that the active state is coherent.

6.5 Lineage Under Revision

Artifact lineage should therefore record at least four relations:

- produced-by: which execution boundary published the artifact
- consumed-by: which downstream boundaries depended on it
- supersedes: which prior artifacts it replaced, if any
- superseded-by: which later artifacts displaced it

This is the durable-state counterpart to execution-lineage replay. Execution lineage tells the runtime when to reuse or recompute work. Artifact lineage tells operators and downstream nodes what maintained state is current, historical, or affected by revision.

7 System Design

7.1 From Blocks and Steps to Artifacts

The sibling ThruWire repository illustrates a practical execution model built around authored blocks, ordered steps, materialized dependencies, and typed local scratchpad state. We abstract away from implementation details and retain the core systems lesson: a runtime that already has explicit blocks, steps, and dependency keys should persist the outputs of those boundaries as structured artifacts rather than collapse them back into transcript state.

In the current design, each block executes against local typed state containing context entries, dependency entries, artifact entries, and step execution records. Dependencies are materialized against resolved input surfaces, not only logical names. This is important because it means artifact identity can depend on both upstream block identity and the concrete input surface under which that block ran. The resulting system distinguishes inherited state from locally produced state, which is exactly the boundary an artifact model needs.

7.2 Dependency References and Input-Sensitive Materialization

One practical lesson from the sibling repository is that dependency references should be input-sensitive. A reference to an upstream block is not enough if that block can run under different resolved inputs. The runtime therefore materializes dependencies against a dependency key that includes both the upstream block identifier and an input hash. The broader abstraction is important for this paper: artifact identity and dependency identity should track the actual state surface consumed downstream, not only an abstract stage name.

This reduces two failure modes common in prompt-centric systems. First, it prevents accidental aliasing of two distinct upstream executions that happen to share a logical name. Second, it prevents downstream reuse from becoming a fuzzy similarity judgment over prompts or prose. If the upstream state changed in a way that matters to the artifact's dependency surface, then the downstream system should know that it is consuming a new object, not merely "roughly the same" context.

7.3 Artifact Roles, Types, and Renderer Surfaces

The underlying abstraction is compatible with multiple payload representations. In ThruWire, the runtime distinguishes artifact roles such as context, transform, decision, action, and output, and it supports typed rendering surfaces rather than treating the final answer as an untyped transcript. The implementation also tolerates incremental or partial artifact construction during streaming. For the paper's purposes, the key abstraction is that the publication boundary for an artifact is renderer- and downstream-facing, not merely conversational.

This matters because the same intermediate artifact may need to support several operations:

- human inspection
- downstream model consumption
- validation against a schema or expected sections
- replay and exact restoration
- supersession by a later artifact

A free-form transcript surface is usually too weak for all of these at once.

7.4 Structured Payloads Beyond Plain Text

The same system may expose artifact payloads in several structured forms. In practice, these may include sectioned markdown, XML-like tagged payloads, key-value structures, or JSON-shaped render payloads. The important point is not allegiance to one serialization format. It is that the artifact boundary should support stable parsing, partial validation, and unambiguous downstream consumption. If one step emits “criteria” and another consumes “criteria,” the consumer should not have to infer where that object begins and ends by rereading a conversation.

This is one reason artifact-first systems can tolerate both human-readable and machine-readable forms. A claim matrix might be displayed as prose for a user while remaining internally addressable as a structured object with claim identifiers, evidence references, exclusions, and status metadata. A recommendation artifact might be rendered as readable text while still preserving explicit slots for constraints, rollout gates, and open questions. The design goal is not maximal formalism. It is preserving enough structure that downstream use and later revision are deliberate rather than heuristic.

7.5 Streaming and Partial State

Artifact-centric systems also change how streaming should be understood. If intermediate artifacts are first-class state, then incremental emission is not only a UI flourish. It is partial construction of a durable object. The sibling repository explicitly treats stream events as persisted state tied to step execution and replay. Again, the abstraction matters more than the product detail: if the platform wants deterministic replay of user-visible execution and artifact publication, then streaming itself participates in the state model.

This is relevant because many contemporary systems implicitly assume a sharp divide between “the live stream” and “the saved result.” In a persistent reasoning system, that divide is weaker. The live stream may expose draft construction of a typed artifact, and replay may need to restore not only the final payload but also the canonical publication sequence that led to it. This does not mean every transient token should become state. It means that once a platform chooses to surface streamed artifact construction as part of its execution contract, it should treat that surface as reconstructable system behavior rather than disposable output noise.

7.6 Why Knowledge Storage Alone Is Insufficient

It might appear that a knowledge base, wiki, or vector store could satisfy the need for persistence. These systems do preserve information, but they usually do not preserve execution identity or invalidation semantics. A wiki entry may say that telehealth expansion should stay budget-neutral; it does not ordinarily say that the implementation-plan artifact and final memo are downstream of a revised criteria artifact and should therefore be recomputed. A knowledge graph can encode semantic relations, but unless it also carries procedural lineage and supersession rules, it is not a complete reasoning substrate.

7.7 Complementarity With Private Chain-of-Thought

The artifact model also clarifies an increasingly important boundary in LLM systems: persistent reasoning does not require exposing private chain-of-thought, nor does it compete with it. The system can preserve deliberate intermediate artifacts while still allowing the model to use raw internal reasoning traces during execution. This is a feature, not a limitation. Private chain-of-thought may be noisy, non-canonical, or inappropriate to expose, but it can still be valuable for local reasoning. What the artifact model adds is an external substrate for the selected intermediate state that the workflow intends to maintain, inspect, improve, and reuse.

8 Evaluation Implications

8.1 Why Evaluation Must Change

The point is not that artifact-first workflows will always write better prose. The central claim is narrower: when work is long-lived and revisable, final-answer quality alone is insufficient because maintained-state quality matters. A suitable evaluation frame should therefore distinguish at least two axes:

- the visible quality of the final answer
- the quality of the maintained artifact state under change

8.2 Illustrative Conditions

To make the distinction concrete, consider three broad classes of systems:

- **Final-answer-only loop.** A loop-centric update that receives the prior final memo and current sources, then produces a revised memo.
- **Loop with manually provided intermediates.** A loop update that also receives prior working notes or manually surfaced intermediates, but still lacks explicit dependency and supersession semantics.
- **Artifact-first DAG workflow.** A workflow with declared stage dependencies, persisted intermediate artifacts, execution identity, and selective recomputation.

The artifact-first condition is not just a prompt variant. It differs at the systems level because graph state itself becomes the maintained artifact substrate under evaluation.

8.3 Illustrative Update Cases

Two generic update cases are especially relevant:

Unrelated-branch update. An adjacent branch changes, but that branch is not upstream of the final output. Correct behavior is to preserve unaffected artifacts exactly and avoid contaminating downstream results with unrelated state.

Intermediate-artifact edit. An upstream decision artifact is revised. Correct behavior is to preserve valid upstream evidence artifacts, supersede the revised intermediate artifact, propagate the change to affected downstream artifacts, and preserve unaffected artifacts.

These tasks expose two distinct state-maintenance capabilities: no-op preservation under irrelevant change, and coherent superseding propagation under relevant intermediate change.

8.4 Metrics

The evaluation emphasizes maintained-state quality. The most relevant metrics are:

- final output quality or faithfulness
- maintained artifact consistency
- exact preservation of unaffected artifacts
- downstream propagation recall for affected artifacts
- traceability to maintained intermediate state
- recomputation and edit localization
- unrelated-branch contamination
- state drift risk under repeated updates

These metrics are appropriate for the thesis of this paper because they operationalize quality of the maintained state rather than only quality of the final memo.

8.5 What Such Evaluation Should Reveal

A final-answer-only evaluation can easily obscure the distinction this paper is trying to make. Several systems may produce a plausible revised memo while differing sharply in what maintained state survives underneath.

In an *unrelated-branch update*, a weak system may unnecessarily regenerate downstream outputs or import unrelated state because it lacks principled unaffected-state preservation. A stronger artifact-centric system should preserve unaffected artifacts exactly and localize change.

In an *intermediate-artifact edit*, several systems may correctly reflect the new constraint in the final answer. A final-answer-only evaluation would therefore treat all of them as successful. The maintained-state metrics should separate them by asking whether upstream artifacts were preserved where appropriate, whether the revised artifact clearly superseded its predecessor, whether the correct downstream artifacts were updated, and whether cross-artifact consistency remained intact.

Two aspects of this distinction are worth emphasizing. First, the artifact-first advantage is not reducible to merely telling the system “what changed.” A system may receive an update event yet still lack principled unaffected-state preservation and downstream invalidation boundaries. Second, manually supplying intermediates is not the same as maintaining artifact state. A loop can be given working notes, but unless those notes are addressable objects with explicit authority and revision semantics, the system still has to reconstruct their role heuristically at update time.

Table 1 summarizes the qualitative contrast.

Condition	Unrelated-branch update	Intermediate-artifact edit
Final-answer-only loop	Regenerates final memo; contamination risk	Final memo updates, but maintained-state consistency is partial
Loop with manual intermediates	Better local guidance, but no explicit invalidation	Intermediate notes help, but supersession remains heuristic
Artifact-first DAG workflow	Preserves unaffected memo exactly	Supersedes criteria and updates downstream artifacts coherently

Table 1: Qualitative evaluation contrast. The central difference is not whether the final memo can be updated once, but whether maintained intermediate state remains coherent under revision.

8.6 Why Final-Answer Quality Alone Is Insufficient

These results support the paper’s central methodological claim. If a workflow is evaluated only on its final memo, then a system that rewrites the memo well but leaves stale or inconsistent intermediate state will score too highly. This is tolerable only when the work is disposable. For long-lived AI work, maintained-state quality matters because future revisions depend on the state that survives the current revision.

In practical terms, this means evaluation should ask not only:

- Did the final answer mention the new constraint?

but also:

- Which artifacts changed?
- Which artifacts remained stable?
- Was a revised intermediate artifact clearly marked as superseding the prior one?
- Which downstream outputs are now affected?
- Can the system localize recomputation instead of globally rewriting work?

8.7 Implications for Longer-Horizon Benchmarks

As agent evaluations increasingly move toward persistent, tool-using, multi-turn settings, they should distinguish *task success at the current turn* from *health of maintained state across turns*. Otherwise, a benchmark may reward systems that repeatedly repair the visible answer while silently accumulating stale or contradictory intermediate state.

An artifact-centric benchmark would therefore track the state graph itself over time. It would measure whether a revised artifact superseded the correct predecessor, whether unaffected branches remained stable, whether downstream

artifacts were updated exactly where necessary, and whether the system could enumerate the affected dependents of an upstream change. These are closer to the operational requirements of long-lived AI work than static answer scoring alone.

9 Discussion

9.1 Intermediate Artifacts Are Not Merely Logs

A common failure mode in current agent systems is to treat intermediate state as either unimportant or purely observational. The transcript is kept as a log; the final answer is kept as the product; everything else is expendable. Our argument is that this underestimates the role intermediate artifacts play in long-lived work. Evidence digests, claim matrices, criteria, plans, and transformation outputs are often the state that later revisions actually need.

This is why the slogan “artifacts are not merely logs” matters. A log describes what happened. An artifact is part of what now exists in the system’s maintained state and can become an object of later improvement.

9.2 A Graph Execution Model Requires an Artifact Model

Work on execution lineage argues that deterministic graphs with explicit dependencies improve replay, recomputation, and update locality. The present paper adds that such a graph execution model is incomplete without a corresponding artifact model. Execution lineage alone tells the runtime what ran. Artifact lineage tells the system what durable state was produced, which of it is current, and what downstream work depends on it. Without artifacts as first-class state, the graph becomes an execution-time convenience rather than a persistent reasoning substrate.

This is why replay and recomputation should not be framed only as execution optimizations. They are state-management operations. A replay hit says that the prior published artifact remains the correct current state under unchanged dependencies. A selective recompute says that a subset of the maintained artifact graph has been invalidated and must be republished. Once phrased this way, the data-model question is unavoidable.

9.3 Why This Matters for Human-AI Collaboration

Although the paper is not primarily a collaboration paper, the artifact model has obvious collaboration implications. Humans can inspect and edit meaningful intermediate boundaries instead of rephrasing high-level instructions into a loop. Other agents can consume the same state with clear expectations about what is authoritative and what has been superseded. This is a more stable collaboration surface than raw conversation history or hidden memory alone.

9.4 Persistent Reasoning Without Full Transparency

The distinction between deliberate artifacts and private chain-of-thought is also practically useful. Some readers may worry that insisting on persistent intermediate state implies exposing internal reasoning traces or abandoning them. We argue neither. The artifact model provides a safer and more controlled complement: the system publishes selected intermediate artifacts that are intended for review and reuse, while keeping raw internal chain-of-thought separate when appropriate. Persistent reasoning therefore does not require making every hidden trace public, but it does benefit from publishing the intermediate artifacts that future reasoning should be able to inspect and improve.

10 Limitations

The proposal has several important limitations.

- **Schema discipline.** Typed artifacts require schema design, publication rules, and discipline. Poor schemas can create brittle systems or encourage superficial structure.
- **Granularity choices.** Artifact boundaries are design decisions. Too coarse, and the system loses update locality. Too fine, and the workflow becomes over-fragmented and costly to manage.
- **Artifact quality.** A persisted artifact can still be incomplete, misleading, or wrong. Durability does not imply truth.
- **Stochastic content.** Even when execution identity is explicit, model-generated artifact content remains sensitive to model behavior and runtime conditions. This is a system-level state model, not a claim of universal model determinism.

- **Not every task needs it.** Short-horizon tasks, disposable chats, and lightweight ideation may not benefit from durable artifact state.
- **Risk of over-structuring.** Excessive formalization can reduce flexibility, especially early in exploratory work where the right intermediate boundaries are not yet clear.

The evaluative discussion is also limited. The examples here are intended to clarify what an artifact-centric evaluation should measure, not to establish universal performance claims across all agentic workflows.

11 Conclusion

This paper provides the corresponding data model for lineage-aware graph execution. If execution lineage answers how the work runs, artifact-centric reasoning answers what durable state that execution should leave behind.

The central claim is that intermediate artifacts are the correct unit of persistent reasoning state for long-lived LLM systems. First-class artifacts are typed, structured, addressable, versioned, dependency-aware, and consumable downstream. They complement transcripts, hidden chain-of-thought, and final answers by providing the inspectable and revisable state surface that those other forms do not provide on their own.

The broader implication is methodological. Persistent AI work should not be judged only by whether the latest answer looks good. It should also be judged by whether the maintained state remains inspectable, reusable, revisable, and coherent over time. If we want to improve reasoning systems rather than merely rerun them, we need intermediate artifacts that can be replayed, inspected, improved, and iterated on across revisions. That is the substrate this paper argues for.

References

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Thomas Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023.
- [2] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, 2023.
- [3] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society. arXiv:2303.17760, 2023.
- [4] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291, 2023.
- [5] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv:2308.00352, 2023.
- [6] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023.
- [7] Max Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, Samy Bengio, and others. Show Your Work: Scratchpads for Intermediate Computation with Language Models. arXiv:2112.00114, 2021.
- [8] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, 2022.
- [9] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations*, 2023.
- [10] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Jason Y. Wei, Tianyi Zhou, Xuezhi Wang, Dale Schuurmans, Claire Cui, Hao Bian, and others. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. In *International Conference on Learning Representations*, 2023.

- [11] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*, 2023.
- [12] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, 2023.
- [13] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, and others. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, 2023.
- [14] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. arXiv:2310.04406, 2023.
- [15] Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-Language Agent Harnesses. arXiv:2603.25723, 2026.
- [16] Yuxuan Zhang, Haoyang Yu, Lanxiang Hu, Haojian Jin, and Hao Zhang. General Modular Harness for LLM Agents in Multi-Turn Gaming Environments. arXiv:2507.11633, 2025.
- [17] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent Workflow Memory. arXiv:2409.07429, 2024.
- [18] Shengda Fan, Xin Cong, Yuepeng Fu, Zhong Zhang, Shuyan Zhang, Yuanwei Liu, Yesai Wu, Yankai Lin, Zhiyuan Liu, and Maosong Sun. WorkflowLLM: Enhancing Workflow Orchestration Capability of Large Language Models. arXiv:2411.05451, 2024.
- [19] Dongge Han, Camille Couturier, Daniel Madrigal Diaz, Xuchao Zhang, Victor Rühle, and Saravan Rajmohan. LEGOMem: Modular Procedural Memory for Multi-agent LLM Systems for Workflow Automation. arXiv:2510.04851, 2025.
- [20] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Hinrich Schütze, Volker Tresp, and Yunpu Ma. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. arXiv:2508.19828, 2025.
- [21] Luiz C. Borro, Luiz A. B. Macarini, Gordon Tindall, Michael Montero, and Adam B. Struck. Memori: A Persistent Memory Layer for Efficient, Context-Aware LLM Agents. arXiv:2603.19935, 2026.
- [22] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks. In *EuroSys*, 2007.
- [23] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster Computing with Working Sets. In *USENIX HotCloud*, 2010.
- [24] Apache Software Foundation. Airflow Documentation: DAGs. <https://airflow.apache.org/docs/apache-airflow/3.0.4/core-concepts/dags.html>, accessed May 6, 2026.
- [25] dbt Labs. dbt Developer Hub. <https://docs.getdbt.com/>, accessed May 6, 2026.
- [26] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and Where: A Characterization of Data Provenance. In *International Conference on Database Theory*, 2001.
- [27] Juliana Freire, David Koop, Emanuele Santos, and Claudio T. Silva. Provenance for Computational Tasks: A Survey. *Computing in Science & Engineering*, 10(3):11–21, 2008.
- [28] Renan Souza, Timothy Poteet, Brian Etz, Daniel Rosendo, Amal Gueroudji, Woong Shin, Prasanna Balaprakash, and Rafael Ferreira da Silva. LLM Agents for Interactive Workflow Provenance: Reference Architecture and Evaluation Methodology. arXiv:2509.13978, 2025.
- [29] Donald E. Knuth. Literate Programming. *The Computer Journal*, 27(2):97–111, 1984.
- [30] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, and others. Jupyter Notebooks – a Publishing Format for Reproducible Computational Workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, 2016.
- [31] Chris Lattner, Jacques Pienaar, Mehdi Amini, Uday Bondhugula, River Riddle, Albert Cohen, Tatiana Shpeisman, Andy Davis, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *International Symposium on Code Generation and Optimization*, 2021.
- [32] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, Jos Lehmann, Roberto Navigli, Axel Polleres, and others. Knowledge Graphs. *ACM Computing Surveys*, 54(4):1–37, 2021.